

Guía DevOps

Carlos Parreño

1. Planificación y Requerimientos (Continuous Planning)

1.1 Estructura del Tablero Kanban (GitHub Projects)

El primer paso es crear un tablero en GitHub con columnas que reflejen el ciclo de vida del desarrollo incremental. Esta estructura conecta el trabajo de documentación con la programación en C#:

- **Backlog (Pila de Producto):** Aquí vivirán todos los requerimientos listados en el documento SRS, priorizados según el método MoSCoW.
+1
- **To Do (Sprint Actual):** Las tareas seleccionadas para desarrollarse en la semana en curso (por ejemplo, la integración de las librerías `plcommpro.dll` y `pltcpcomm.dll`).
- **In Progress:** Tareas que los 4 programadores están codificando activamente.
- **In Review / QA:** Tareas que ya fueron programadas y están esperando revisión de código o validación de los Criterios de Aceptación (CA).
- **Done:** Requerimientos validados, fusionados en la rama `main` y listos para despliegue.

1.2 Traducción de Requerimientos a "Issues"

Un requerimiento en un documento de Word es estático; en DevOps, debe convertirse en un "Issue" vivo dentro de GitHub. Ustedes deben crear una plantilla estándar para que los programadores tengan el contexto exacto.

Ejemplo de cómo transformar un requerimiento del SRS a un Issue:

- **Título del Issue:** [MUST] RF-07: Autenticación institucional mediante Microsoft SSO
- **Descripción (User Story):** Como usuario del sistema (Docente, Estudiante o Administrativo), quiero iniciar sesión utilizando mi correo institucional mediante Microsoft SSO para acceder a mi portal de autoservicio.
+2
- **Criterios de Aceptación (Checklist para QA):**
 - ☐ El sistema redirige al portal de Microsoft Server.
 - ☐ Cuando un usuario institucional inicia sesión con Microsoft, el sistema crea/actualiza su perfil.
 - ☐ El sistema permite el acceso y asigna permisos según el rol correspondiente.

1.3 Gestión de Prioridades (El modelo MoSCoW en acción)

El uso de etiquetas (tags) en GitHub es crucial para mantener la metodología incremental. Asignen las siguientes etiquetas a sus Issues basándose en su levantamiento:

- **priority: MUST (Crítico para el lanzamiento):** * *Ejemplo:* Controlar plumas: apertura automática por validación y apertura manual por usuario Garita con logs de auditoría.
 - *Ejemplo:* Restringir a máximo 2 vehículos por usuario institucional.
- **priority: SHOULD (Importante, pero no bloqueante en la fase 1):** * *Ejemplo:* Configurar tarifas variables por tipo de automotor, discapacidad y mixto.
- **priority: PENDIENTE (Para futuras iteraciones):** * *Ejemplo:* Modo visitante.

1.4 El Flujo de Trabajo (Documentación ↔ Desarrollo)

Para que el Continuous Planning funcione, el flujo de comunicación debe ser estricto:

1. **Refinamiento (Ustedes 3):** Toman el acta semanal y el SRS, crean los Issues en GitHub y definen los Criterios de Aceptación.
2. **Asignación (Los 4 Programadores):** Un programador se asigna a sí mismo un Issue de la columna "To Do", lo mueve a "In Progress" y crea su rama de Git (ej. `feature/RF-17-control-plumas`).
3. **Validación (Trabajo conjunto):** Cuando el programador termina, abre un Pull Request. Alguien del equipo de documentación revisa que el código cumpla exactamente con los Criterios de Aceptación definidos antes de aprobar la fusión.

2. Integración Continua (CI)

2.1 Proteger la rama main (Regla de Oro)

Antes de automatizar, ustedes (el equipo de documentación y gestión) deben configurar las reglas del repositorio en GitHub para proteger el código de producción.

- **Acción:** Vayan a *Settings > Branches* en su repositorio de GitHub y agreguen una regla (Branch protection rule) para la rama `main`.
- **Configuración:** Activen la opción *"Require a pull request before merging"* y *"Require status checks to pass before merging"*.
- **¿Qué logran con esto?:** Ningún programador podrá enviar código directamente al `main`. Estarán obligados a crear una rama alterna, hacer un *Pull Request* (PR), y esperar a que GitHub Actions confirme que el código no tiene errores antes de que ustedes aprueben la fusión.

2.2 Crear el Pipeline de GitHub Actions (El archivo YAML)

GitHub Actions funciona mediante archivos de configuración escritos en formato YAML. Este archivo le dice a los servidores de GitHub exactamente qué hacer con el código.

1. En la raíz de su repositorio, creen una carpeta llamada `.github/workflows/`.
2. Dentro de esa carpeta, creen un archivo llamado `ci-dotnet.yml`.

2.3 El Script de Integración (.NET 8)

Aquí hay un detalle de ingeniería crítica para su proyecto: dado que su prototipo incluye Windows Forms y librerías DLL, el entorno virtual ("runner") que compile el código **debe ser Windows**, no Linux.

Este es el script base que deben pegar en su archivo `ci-dotnet.yml`:

YAML

```
name: Integracion Continua Parquaderos

# El script se ejecuta cuando hay un Pull Request hacia la rama main
on:
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    # Usamos Windows porque dependen de librerías .dll para el inBio 260
    runs-on: windows-latest

    steps:
      # Paso 1: Descargar el código del repositorio
      - name: Checkout del código
        uses: actions/checkout@v4

      # Paso 2: Instalar el entorno .NET 8 (v8.0.41x)
      - name: Configurar .NET 8
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      # Paso 3: Restaurar paquetes críticos (SqlClient, BCrypt, etc.)
      - name: Restaurar dependencias
        run: dotnet restore

      # Paso 4: Compilar el proyecto para verificar que no hay errores de sintaxis
      - name: Compilar el proyecto
        run: dotnet build --no-restore --configuration Release
```

2.4 El Flujo de Trabajo en la Práctica (Documentación ↔ Desarrollo)

Con este archivo configurado, el flujo de trabajo diario de su equipo incremental será el siguiente:

1. **El Desarrollo:** Un programador toma el Issue "*RF-08 Registrar vehículos*", crea su rama, escribe el código y hace un *Push*.
2. **La Automatización (CI):** Al abrir el *Pull Request*, GitHub Actions lee el archivo YAML. Automáticamente levanta una máquina virtual con Windows, instala .NET 8 , restaura paquetes como SqlClient y BCrypt, e intenta compilar el código.
3. **La Validación (Ustedes):** En la pantalla del *Pull Request*, ustedes verán un indicador:
 - **Check Verde** : El código compiló perfectamente y no rompió nada. Ustedes pueden proceder a revisar la lógica del negocio.
 - **Equis Roja** : El código tiene un error de compilación. El programador debe arreglarlo en su máquina y volver a subirlo. Ustedes no tienen que perder tiempo revisando código roto.

2.5 Recomendación para la Arquitectura Multisitio

Dado que el proyecto tiene un backend (que eventualmente irá a un servidor) y un módulo local de Garita (Windows Forms), en iteraciones futuras pueden separar este archivo YAML en dos flujos distintos: uno que compile la API web en un entorno Linux (para prepararlo para Docker) y otro que compile el ejecutable de Garita en Windows.

3. Pruebas de Seguridad Continua (DevSecOps)

3.1 Análisis de Composición de Software (SCA) con Dependabot

Sus programadores están restaurando paquetes críticos como `SqlClient` y `BCrypt` para la seguridad y persistencia de datos. Si el día de mañana se descubre que la versión específica de `BCrypt` que están usando tiene una falla que permite hackear contraseñas, ustedes deben enterarse de inmediato.

- **La Herramienta: GitHub Dependabot** (Nativo y gratuito en GitHub).
- **Cómo se aplica (El trabajo de Documentación/Gestión):**
 1. Vas a la pestaña **Settings** de tu repositorio en GitHub.
 2. Entrás a **Code security and analysis**.
 3. Activas **Dependabot alerts** y **Dependabot security updates**.
- **El Flujo:** Dependabot leerá automáticamente sus archivos de proyecto de C#. Si encuentra que una librería está desactualizada o tiene una vulnerabilidad conocida (CVE), Dependabot creará un *Pull Request* automático subiendo la versión de la librería. Ustedes solo tendrán que revisar que los tests pasen (Fase 2) y darle a "Merge".

3.2 Pruebas Estáticas de Seguridad (SAST) con SonarCloud

Dependabot revisa el código de *otros* (las librerías), pero necesitamos revisar el código en C# que escriben *sus 4 programadores*. Especialmente porque manejarán integraciones críticas como el inicio de sesión único (SSO) de Microsoft y no queremos que dejen tokens o contraseñas quemadas en el texto plano del código.

- **La Herramienta: SonarCloud** (Tiene plan gratuito para proyectos de estudiantes/código abierto) o **CodeQL** (Nativo de GitHub). Te recomiendo SonarCloud porque su interfaz visual es excelente para documentadores.
- **Cómo se aplica:**
 1. Creas una cuenta en vinculando tu GitHub.
 2. Importas el repositorio del proyecto de Parquederos.
 3. SonarCloud te dará un par de "Tokens" (contraseñas secretas). En GitHub, vas a **Settings > Secrets and variables > Actions** y guardas esos tokens ahí.

3.3 Integración en el Flujo de Trabajo (El YAML)

Para que esto sea verdaderamente DevSecOps, SonarCloud debe ejecutarse dentro de la Integración Continua (CI) que configuramos en el paso anterior.

Los programadores simplemente deben agregar un par de líneas al archivo `ci-dotnet.yml` para que, antes de compilar el código .NET 8, el escáner lo lea:

3.4 Documentación y Requerimientos

Mediante DevSecOps:

1. Cuando un programador hace un *Pull Request*, GitHub ejecutará el YAML.
2. SonarCloud publicará un comentario automático directamente en GitHub diciendo algo como: *"Se encontraron 3 Code Smells y 1 Vulnerabilidad Mayor"*.
3. **El trabajo:** Revisar ese reporte. Si SonarCloud marca que el código no pasa el "Quality Gate" (Filtro de Calidad), ustedes le rechazan el *Pull Request* al programador y le crean un nuevo *Issue* en el tablero Kanban que diga: *"Corregir vulnerabilidad de inyección SQL en el módulo de Garita"*.

De esta forma, nunca llega código inseguro a la rama `main` y ustedes mantienen un control de calidad estricto sobre el ciclo de desarrollo.

4. Despliegue Continuo (CD) y Contenerización

4.1 La Estrategia Arquitectónica (La División)

Tenemos dos componentes principales que se despliegan diferente:

- **El Módulo de Garita (Windows Forms):** Como este módulo necesita conectarse al hardware (el controlador inBio 260) y usa librerías nativas de Windows (`plcommpro.dll` y `pltcpcomm.dll`), **este componente NO va en Docker**. Su despliegue debe ser un instalador (un `.exe`) que se coloca directamente en la computadora física de la garita.
- **El Backend, Base de Datos y Panel Web:** Todo lo que maneja el inicio de sesión con Microsoft (SSO) , el cálculo de tarifas y los reportes administrativos. **Este componente SÍ va en Docker**.

4.2 ¿Qué es Docker y cómo se usa aquí?

Imagina que Docker es un contenedor de transporte marítimo. Adentro metes tu aplicación web, el entorno de .NET 8, y todas las configuraciones. Ese contenedor se puede ejecutar en la laptop de cualquiera de los 4 programadores o en el servidor final de la PUCE, y funcionará *exactamente* igual.

La herramienta principal: El Dockerfile Los programadores crearán un archivo de texto llamado `Dockerfile` en el repositorio. Es simplemente una receta. Se ve algo así:

4.3 El Flujo de Despliegue Continuo (CD) en GitHub Actions

Ahora conectamos lo que hicimos en la fase anterior (CI) con esta nueva fase (CD). El objetivo es que, cuando ustedes aprueben un *Pull Request* y el código llegue a la rama `main`, el sistema se actualice solo en el servidor.

Se añade un "Job" extra al archivo YAML (`ci-dotnet.yml`) que ya teníamos:

Una vez que la imagen se sube a la nube, el servidor de la universidad solo tiene que descargarla y reiniciar el sistema. ¡Despliegue completado en minutos y sin intervención humana!

4.4 Fase de Despliegue (Documentación)

Como documentadores y gestores, su trabajo en esta fase de CD no es escribir el Dockerfile, sino controlar el entorno:

1. **Gestión de Variables de Entorno (Secrets):** El sistema necesita contraseñas de la base de datos y las llaves secretas del SSO de Microsoft. Ustedes deben asegurar que esas llaves nunca se escriban en el código fuente ni en los requerimientos, sino que se guarden de forma segura en los *Secrets* de GitHub.
2. **Documentación de la Infraestructura:** Deben crear un diagrama de despliegue que muestre cómo el servidor web se conecta a la computadora de la garita.
3. **Release Notes (Notas de la Versión):** Cada vez que se hace un despliegue exitoso desde la rama `main`, ustedes redactan qué requerimientos (ej. *RF-01*, *RF-02*) se acaban de publicar para que el director del proyecto y los interesados (stakeholders) sepan qué pueden empezar a probar.